RESEARCH VALIDATION REPORT

Phase 4E
Literature Review & Novelty Assessment

AI-Driven RTL Test Generation, Coverage Analysis and Verification

Simulation-only M.Tech study | UART TX/RX case study | Phase-based development workflow

Evidence date: 10 October 2026 | Scope: verified UART trials, fault injection, coverage feedback and archived pre-layout PPA

Research decision

The four-phase implementation and its 12/12 acceptance audit are complete. The work supports a bounded, experimentally evidenced M.Tech contribution, but the underlying LLM test-generation, AST analysis and coverage-feedback techniques already have substantial published prior art. Publication-level novelty has not been established.

Sources: user-supplied WSL execution outputs and the archived base-paper preview, compared with verified publisher/DOI metadata. A separate WSL evidence-audit script checks original JSON results before thesis submission.

1. Project scope and research questions

The implemented simulation-only pipeline is: Phase 1 UART baseline and optimized RTL; Phase 2 PyVerilog AST to UART knowledge; Phase 3 Hermes structured scenarios to deterministic validator, self-checking Icarus testbench, coverage and feedback; Phase 4 reproducibility, pre-layout PPA analysis, mutation testing and reporting. ALU, FIFO and FSM earlier technical work is retained as supporting evidence rather than conflated with UART-specific AI measurements.

Research question	Evidence and appropriate interpretation
RQ1: Can a real LLM generate valid UART tests?	Five fresh valid Hermes generations and one archived initial generation; deterministic JSON constraints and simulator checks.
RQ2: How does coverage compare with fair baselines?	Compare equal initial 10-scenario budgets, but also disclose differences in simulation cycles; balanced-directed achieved 100%.
RQ3: Do tests detect faults?	14 chosen fault operators, four CPBs, corrected reset-cycle oracle; AI and balanced-directed each killed all 14.
RQ4: Does feedback close a measured gap?	Trial 5 edge_81 was 90%; one explicit Hermes follow-up plus a matched directed follow-up each reached 100%.
RQ5: Does UART implementation efficiency improve?	Archived pre-layout mapped architecture comparison is separate from verification AI: best hybrid lowered area and estimated power.

Evidence basis: accepted WSL terminal logs. The supplied project does not establish results on independent third-party RTL designs.

2. Verified quantitative evidence

2.1 Functional coverage and independent repetitions

Method	Runs	10-bin coverage	Important qualifier
Payload-only random	10 seeds	Mean 13%; range 10–30%	Weak baseline; mostly loopback stimuli
Protocol-aware random	10 seeds	Mean 69%; range 50–80%	Stronger generated protocol scenarios
Balanced directed	1 suite	100%	A well-designed suite matches AI
Fresh Hermes	5 independent calls	Mean 98%; range 90–100%; 4/5 full	Five distinct scenario sequences; one 90% result
Original Hermes	1 call	100%	Separate archived evidence

All five fresh Hermes trials produced valid scenario sets and passed 20 total Icarus runs across CPB 8/16/32/64, with 182,040 reported checks and no failures. Fresh model calls should not be equated with statistically independent samples unless provider/model settings and sampling controls are recorded.

2.2 Corrected checker and mutation sensitivity

Verification method	Old checker	Corrected checker
Hermes AI (five fresh suites)	92.86%	100.00%
Hermes archived	92.86%	100.00%
Balanced directed	92.86%	100.00%
Protocol-aware random (10)	77.14%	81.43%
Payload-only random (10)	50.71%	50.71%

The final corrected-oracle study passed 108 clean controls and 1,512 mutant simulations, with zero invalid runs. The previously surviving TX_RESET_BUSY_FAULT mutant exposed a reset-observability weakness in the checker; correcting the assertion changed the score without changing AI scenarios. Thus the measured gain is partly due to verification-oracle quality, not solely stimulus generation.

2.3 Targeted feedback and architecture data

For the fifth Hermes trial, the only unhit functional bin was edge_81. An additional validated Hermes-generated loopback(0x81) scenario changed coverage from 90% (10 scenarios) to 100% (11). A matched directed scenario achieved exactly the same closure. The feedback prompt disclosed both the missing bin and the deterministic loopback(129) remedy; this is feedback-loop integration, not autonomous discovery.

Mapped UART arrangement	Cell area (μm²)	Estimated power (μW)
Baseline TX + baseline RX	450.604	370.282
Optimized TX + optimized RX	440.230	392.761
Baseline TX + optimized RX (hybrid)	425.600	366.547

These synthesis mapping values are pre-layout estimates, not measured silicon power or an AI-created architecture improvement. The hybrid reduces area by about 5.55% and estimated power by about 1.01% versus the baseline. Results are archived separately from the AI verification evaluation.

3. Published research and closest methodological prior art

The references below were checked against IEEE/ACM/institutional, Springer and Elsevier publisher or DOI records as available by 10 October 2026. Short source descriptions are based on the public abstracts/metadata and the supplied Elsevier preview, not a full-text systematic review.

Study	Prior contribution	Position of this UART project
[R1] AutoBench, 2024	LLM-driven HDL testbench generation and evaluation.	Overlap: test generation; contribution here is a constrained UART simulator/coverage/mutation evaluation.
[R2] CorrectBench, 2025	Automatic self-validation and correction of generated testbenches.	Overlap: simulation feedback; here AI emits whitelisted scenarios, not unrestricted HDL code.

[R3] LLM4DV, 2025	Hardware test stimuli targeting predefined coverage bins.	Direct overlap: bin-oriented LLM stimuli; do not claim this concept is new.
[R4] ChatTest, 2026	Coverage-aware test augmentation using an LLM feedback loop, across 20 RTL designs.	Direct overlap: closed-loop RTL verification; ChatTest tests broader RTL scope.
[R5] AssertMiner-pro, 2026	RTL AST and hierarchical LLM-assisted module assertions and mutation-based evaluation.	Conceptual base paper, but different generated artifact: assertions versus UART test stimuli.
[R6] SearchSYS, 2025	LLM-generated program seeds plus software fuzzing for gem5 system simulator.	Methodological reference only; not RTL protocol verification.
[R7] ETS survey, 2025	Overview of LLM applications to hardware verification/testing/design.	Background and known limitation framing; not a directly comparable benchmark.

4. How the supplied base paper differs from the implementation

The uploaded “Base paper(1).pdf” is a six-page print/preview of the Elsevier AssertMiner-pro article [R5]. The accessible portion covers its abstract, introduction, contributions and approach; full article experiments must be checked before attributing detailed quantitative conclusions.

Dimension	AssertMiner-pro [R5]	This M.Tech implementation
Primary problem	Coverage of module-level properties and deep assertions	Automated test stimuli, functional coverage gaps and UART behavior
AST role	Structure and top-down hierarchical guidance for module specifications	PyVerilog parsing of TX/RX structural and behavioral knowledge
LLM output	Module-level specifications and fine-grained assertions	JSON scenario list with whitelisted operations and byte values
Execution/checking	Assertion-centric verification and mutation evidence	Icarus self-checking differential testbench, reset-cycle oracle and mutants
Experimental scope	Multiple design/module cases in publisher abstract	UART case across four CLKS_PER_BIT settings
How to describe relation	Conceptual methodological inspiration	Not a reproduced implementation or numerical improvement over AssertMiner-pro

For a review presentation, describe [R5] as the selected base paper for AST-assisted verification, then explain the deliberate change in scope from assertion mining to structured UART stimulus generation and executable coverage feedback. Do not state that the project extends AssertMiner-pro source code unless that modification was actually made and documented.

5. Novelty decision and defensible contribution statements

5.1 Supported contributions

- A functioning phase-integrated UART flow from AST-derived RTL knowledge to constrained LLM-generated scenarios, self-checking simulation, functional coverage and targeted feedback.
- Five fresh Hermes trials with a 98% mean on ten defined UART coverage bins, alongside ten-seed weak and protocol-aware random controls and a 100% balanced-directed control.
- Fault-sensitivity evidence on 14 intentionally injected RTL faults at four clock configurations, including an independently diagnosed and corrected reset-cycle observation gap.
- A measured 90% to 100% coverage closure using one actual Hermes feedback scenario, with an identical-capability directed control and explicit incremental budget.
- Separate technology-mapped UART architecture/PPA analysis with a lower-area/lower-estimated-power hybrid, without falsely attributing it to AI.

5.2 Claims that must NOT be made

- “First-ever LLM coverage feedback loop for RTL” - ChatTest and LLM4DV establish related prior art.
- “AI outperforms expert-directed testing” - the balanced-directed suite matched 100% bin coverage and 100% corrected mutation score.
- “100% RTL verification/bug coverage” - only ten selected functional bins and fourteen chosen mutant operators were evaluated.
- “AI reduced silicon area or measured chip power” - the PPA comparison is a separate pre-layout architecture experiment.
- “Universally effective / statistically proven” - one UART design, five model repetitions, prompt-guided closure and varying simulation cycles limit inference.

5.3 Suggested thesis contribution wording

“This work implements and evaluates a simulation-only, coverage-aware UART RTL verification workflow that converts PyVerilog structural knowledge into constrained Hermes-generated scenarios, executes deterministic self-checking Icarus simulations, identifies functional coverage gaps, and integrates targeted feedback. Across five fresh model calls, the method achieved a mean of 98% coverage on ten selected functional bins. With a corrected reset-cycle oracle, AI-generated and balanced-directed suites each detected all fourteen selected mutants over four clock settings. The framework demonstrates reproducible automated test orchestration and gap closure, while showing comparable rather than superior performance to a carefully designed directed baseline.”

6. Limitations, threats to validity and next decisions

Threat	How it affects interpretation / next requirement
Single UART benchmark	Generalization needs additional unrelated DUTs or standard benchmarks (ALU/FIFO/FSM tested with same current toolchain).
Ten enumerated functional bins	Add realistic line/branch/toggle and protocol transition metrics where tools can measure them.

Oracle dependence	Include independent reference/specification checks, reset-cycle probes, and false-positive controls.
Selected mutants	Expand mutation taxonomy and operator distribution; acknowledge manually selected operators.
Experimental budgets	Test counts equal but cycles/LLM time differ; record wall time, tokens, cost, validity and simulation effort.
Model/provider variability	Record provider, exact model revision, temperature, prompts, seed availability and generation failures.
Prior art overlap	Do not assert firstness; obtain complete ChatTest, LLM4DV and AssertMiner-pro methods for direct feature comparison.
PPA limitations	Pre-layout library-dependent estimates; no physical design extraction, tape-out, FPGA or on-chip power.

7. Review presentation sequence and remaining phases

Recommended five-minute progression: (1) motivating problem in RTL verification; (2) explain the four phase workflow; (3) connect the base paper to AST extraction but distinguish assertion outputs; (4) present three baseline types plus independent Hermes repeatability; (5) show reset-observability fault and corrected mutation results; (6) demonstrate measured Trial 5 closure with matched directed control; (7) finish with scoped contribution and future work. This avoids day-wise storytelling.

Remaining review milestone	Acceptance evidence
Phase 4E - evidence trace	Execute the supplied read-only script; save source-specific JSON and Markdown trace in phase_04_uart_novelty/research/.
Phase 4F - thesis / slides	Revise literature chapter, experiments, novelty and limitations; explain phases, not days; verify diagrams against code.
Phase 4G - reproducibility	Run clean checkout/tool tests; verify SHA manifests, remove sensitive data and check library redistribution terms.
Publication decision	Optional independent-benchmark study and comparison to nearest methods before claiming conference/journal novelty.

8. Phase-based evidence locator

Finding	Existing WSL evidence path (relative to phase_workflow)
12/12 workflow audit	evaluation/workflow_status.json
Initial Hermes and baseline coverage	phase_03_ai_verification/results/evaluation_summary.json
10-seed baseline statistics	evaluation/repeatability/phase4c_*/phase4c_summary.json
Five fresh Hermes generations	evaluation/repeatability/hermes_trials/phase4c2_*/phase4c2_summary.json

Corrected 14-mutant evaluation	evaluation/fault_injection/phase4d5_reset_checker_*/ corrected_checker_summary.json (1512 runs)
Hermes feedback closure	evaluation/feedback_closure/phase4d7_*/ experiment_status.json
Archived PPA	phase_04_uart_novelty/UART_ARCHITECTURE_RESULTS.md

Globs (*) represent timestamped evidence directories. The supplied Phase 4E audit resolves and verifies the relevant successful records without modifying original results.

References: verified bibliographic records

[R1] Qiu, R.; Zhang, G. L.; Drechsler, R.; Schlichtmann, U.; Li, B. (2024). AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design. ACM/IEEE MLCAD. DOI: https://doi.org/10.1145/3670474.3685956.

[R2] Qiu, R.; Zhang, G. L.; Drechsler, R.; Schlichtmann, U.; Li, B. (2025). CorrectBench: Automatic Testbench Generation with Functional Self-Correction using LLMs for HDL Design. IEEE DATE. DOI: https://doi.org/10.23919/DATE64628.2025.10992873.

[R3] Zhang, Z.; Szekely, B.; Gimenes, P.; Chadwick, G.; et al. (2025). LLM4DV: Using Large Language Models for Hardware Test Stimuli Generation. IEEE FCCM. DOI: https://doi.org/10.1109/FCCM62733.2025.00048.

[R4] Wan, G.-W.; Su, S.; Zhang, J.; Wong, S.-Z.; Xing, M.; et al. (2026). ChatTest: Coverage-Enhanced Testbench Generation for Agile Hardware Verification with LLMs. IEEE DATE. DOI: https://doi.org/10.23919/DATE69613.2026.11539733.

[R5] Wang, Y.; Zhou, J.; Lyu, H.; Chen, B.; et al. (2026). AssertMiner-pro: Enhanced module-level spec generation and assertion mining with LLM guided by top-down hierarchical strategies. Integration (Elsevier), vol. 110, article 102797. DOI: https://doi.org/10.1016/j.vlsi.2026.102797.

[R6] Dakhama, A.; Even-Mendoza, K.; Langdon, W. B.; Menéndez, H. D.; Petke, J. (2025). Enhancing search-based testing with LLMs for finding bugs in system simulators. Automated Software Engineering (Springer), 32:63. DOI: https://doi.org/10.1007/s10515-025-00531-7.

[R7] Jha, C. K.; Hassan, M.; Qayyum, K.; et al. (2025). Large Language Models (LLMs) for Verification, Testing, and Design. IEEE European Test Symposium (ETS). DOI: https://doi.org/10.1109/ETS63895.2025.11049311.

Bibliographic status: DOI/venue/title verified from publisher, institutional or reliable index sources. The user-supplied AssertMiner-pro preview is not the publisher's complete downloadable article. The verification claims in this report are based on the user's shared terminal outputs unless separately regenerated by the WSL evidence audit.